

Guía de Sustentación — Regresión, Fluent Assertions y CI del Módulo Bolsillos

Asignatura	Validación y Verificación de Software
Proyecto	FuBanking — Banco digital
Alcance	Bolsillos (5 funcionalidades) + Depositar dinero (módulo Cuentas)
Rama	tests/bolsillos-regresion-fluent (sobre devops/jenkins-gitops)
Documento técnico de soporte	docs/pruebas-regresion-bolsillos.{md,docx,pdf}
Fecha	2026-09-29

1. Qué hice, en 30 segundos

- Elegí **seis funcionalidades**: crear, consultar, actualizar, eliminar y transferir bolsillos, más **depositar dinero**. Revisé el repositorio para que ninguna la estuviera trabajando un compañero.
- Construí una **suite de regresión de 94 casos** que prueba las seis **a través de la API HTTP real** con supertest, siguiendo el patrón que usa el equipo en autenticación (`createTestApp`).
- Escribí todas las pruebas con **aserciones fluidas** (Chai BDD, incluido en Vitest). Convertí las 6 suites unitarias existentes de Bolsillos (58 pruebas) y escribí 12 unitarias nuevas para el depósito.
- Demostré que la suite verifica el funcionamiento después de un cambio**:
 - Ejecuté las mismas **198 pruebas** sobre el código de antes del refactor y sobre el actual: 0 diferencias.
 - Rompí el código a propósito y la suite lo detectó.
- Monté el trabajo **sobre la rama del pipeline de Jenkins**, que lo ejecuta, manda la cobertura a **SonarQube** y construye y levanta la app con **Docker**.

2. Mapa rúbrica → qué hice → cómo lo demuestro

Criterio (puntos)	Qué hice	Evidencia que muestro
Integración Continua (30)	Mi rama parte de <code>devops/jenkins-gitops</code> . Su pipeline hace build, pruebas (incluye las mías), SonarQube, Quality Gate, Docker build, smoke test con Docker, push y actualización del repo GitOps.	Ejecución en Jenkins: etapa <i>Backend: install + test + build</i> con <code>regression/pocket</code> y <code>regression/account</code> ; dashboard de SonarQube; etapa <i>Run with Docker</i> .
Fluent Assertions (20)	70 unitarias + 94 de regresión en Chai BDD, que cubren las seis funcionalidades .	<code>CreatePocket.test.ts</code> antes/después, <code>DepositMoney.test.ts</code> , <code>crear.regression.test.ts</code> ; <code>npm run test:regression:bolsillos</code> .
Pruebas de regresión (20)	94 casos de API; comparación antes/después (198/198) y demo en vivo de cambio → rojo → arreglo → verde.	Sección 5 (demo) y <code>docs/evidencia-regresion-bolsillos/</code> .

Criterio (puntos)	Qué hice	Evidencia que muestro
Presentación (30)	Esta guía: guion, comandos y respuestas.	Secciones 4 a 9.

3. Conceptos que debo dominar

Concepto	Explicación corta (para decirla en voz alta)
Prueba de regresión	Prueba que se vuelve a ejecutar después de cada cambio para confirmar que lo que ya funcionaba sigue funcionando .
Fluent assertions	Aserciones que se encadenan y se leen como una frase : <code>expect(x).to.have.property('balance', 800000)</code> . El nombre viene de <i>FluentAssertions</i> de .NET; en JavaScript el equivalente estándar es Chai BDD , que ya viene en Vitest.
Nivel API (supertest)	Peticiones HTTP reales a una app Express armada para la prueba. Pasan por JWT real → validador → controlador → caso de uso → manejador de errores . Solo la BD va en memoria.
Fake con semántica de copia	Repositorio en memoria que guarda copias , como una BD. Si el código olvida persistir un cambio, la prueba falla.
Invariante de dinero	<code>saldo disponible + Σ bolsillos</code> no cambia al crear, actualizar, eliminar o transferir (solo se mueve el dinero). El depósito lo aumenta exactamente en el monto.
<code>it.fails</code>	Prueba que afirma la regla correcta de un defecto conocido y hoy falla. La suite sigue en verde; cuando alguien lo corrija, se pone roja para avisar. Hay 5: D-01/03, D-02, D-04, D-05 y D-08.
AAA / 5 dobles	Arrange–Act–Assert en cada prueba. Las unitarias del depósito usan Dummy, Fake, Stub, Spy y Mock.
Mutante equivalente	Cambio de código que no altera el comportamiento, así que ninguna prueba puede detectarlo (ejemplo en la sección 5).

4. Guion de presentación (≈ 10 minutos)

Min	Qué digo	Qué muestro
0–1	Las 6 funcionalidades y por qué el depósito es la sexta (nadie más la tenía; amplía el saldo que se aparta en bolsillos).	Tabla 2.1 del documento técnico.
1–3	Fluent assertions : qué son, por qué Chai BDD, ejemplo antes/después.	<code>CreatePocket.test.ts</code> , <code>DepositMoney.test.ts</code> .
3–5	Suite de regresión : nivel API, qué es real y qué es fake, escenario, qué verifica cada caso.	<code>createPocketTestApp.ts</code> , <code>scenario.ts</code> , <code>depositar.regression.test.ts</code> .
5–6	Ejecutar: 193 aprobadas + 5 defectos conocidos .	<code>npm run test:regression:bolsillos</code> .
6–8	Demo en vivo: cambio → rojo → arreglo → verde .	Sección 5.
8–9	Antes vs después : 198/198 iguales.	Documento técnico §7.
9–10	Pipeline : dónde entran mis pruebas, SonarQube y Docker.	Jenkins.

5. Demo en vivo: cambio → rojo → arreglo → verde

Paso 1 — suite en verde:

```
cd backend
npm run test:regression:bolsillos
# Tests 193 passed | 5 expected fail (198)
```

Paso 2 — introducir un error en `backend/src/application/use-cases/pocket/DeletePocket.ts`:

```
// original
await this.accountRepository.updateBalance(account.id, account.balance + pocketAmount);
// con el error (resta en vez de sumar)
await this.accountRepository.updateBalance(account.id, account.balance - pocketAmount);
```

Paso 3 — la suite lo detecta:

```
x elimina el bolsillo y devuelve su monto al saldo de la cuenta (unitaria)
x RG-EL-02 · borra el bolsillo y devuelve su monto al saldo disponible (regresión API)
x RG-EL-06 · un segundo DELETE responde 404 y no devuelve el dinero dos veces
x RG-FL-01 · crear → consultar → actualizar → transferir → eliminar conserva el total
Tests 4 failed | 189 passed | 5 expected fail (198)
```

Paso 4 — revertir (`git checkout -- src/application/use-cases/pocket/DeletePocket.ts`) y volver a ejecutar: **193 passed | 5 expected fail**.

Alternativa con la sexta funcionalidad: en `DepositMoney.ts`, comentar `account.assertBelongsTo(dto.userId);`. Fallan **2 pruebas**: la unitaria de FORBIDDEN y RG-DE-12, porque cualquiera podría depositar en una cuenta ajena.

Otros cambios verificados:

Cambio	Resultado
<code>TransferPocketBalance</code> : no guardar el bolsillo destino	3 fallan
<code>UpdatePocket</code> : > por >= en la validación de saldo	RG-AC-07 falla (valor límite)
<code>DepositMoney</code> : <= 0 por < 0	Ninguna falla, y es correcto: <code>!dto.amount</code> ya rechaza el 0 (mutante equivalente)

6. Evidencia antes vs después

- **Antes:** 6102a28, código previo a cualquier refactor.
- **Después:** rama del pipeline (`devops/jenkins-gitops`, 3205dc1). Incluye:
 - el refactor de `UpdatePocket` (cognitiva 16 → 6);
 - `node:crypto` en los casos de uso;
 - `z.uuid()` de zod 4 en los validadores.

- **Procedimiento:** `git worktree` con el código viejo + **el mismo paquete de pruebas**, ejecución con reporte JSON y comparación caso por caso.
- **Resultado:** 18 archivos, **198 casos, 0 diferencias** (193 aprobados + 5 `it.fails` en ambos).
- **Archivos:** `docs/evidencia-regresion-bolsillos/antes-6102a28.txt`, `despues-rama-pipeline.txt`, `comparacion.json` y `diff-produccion-antes-despues.patch`.

7. Integración continua

Checkout → Backend: `install + test + build` → Frontend: `install + test + build`
 → SonarQube analysis → Quality Gate → Docker build → Run with Docker (smoke test)
 → Docker push → Update GitOps repo (FuBanking-gitops) → ArgoCD sincroniza en minikube

Pregunta	Respuesta
¿Dónde corren mis pruebas?	En <i>Backend</i> : <code>install + test + build</code> (<code>npm run test:coverage</code>). Vitest incluye <code>src/tests/**/*.test.ts</code> , así que entran solas.
¿Necesitan BD en CI?	No. El pipeline crea un <code>.env</code> con <code>JWT_SECRET</code> , que es lo único que necesitan.
¿Cómo llegan a SonarQube?	<code>test:coverage</code> genera <code>backend/coverage/lcov.info</code> , que lee el scanner.
¿Y Docker?	Tras el Quality Gate se construyen las imágenes, se levantan en una red de Docker, se verifica <code>/health</code> y se publican; el repo FuBanking-gitops recibe el nuevo tag y ArgoCD despliega.
¿Pasa localmente como en CI?	Sí: 64 archivos, 498 aprobadas + 5 <code>it.fails</code> , 99,6 % de cobertura, <code>tsc</code> sin errores.
¿Por qué la rama parte de <code>devops/jenkins-gitops</code> y no de <code>main</code> ?	Porque el <code>Jenkinsfile</code> y los <code>Dockerfile</code> solo existen en esa rama. Además, allí ya está corregido el cableado del depósito (en <code>main</code> se pasaba el repositorio de transacciones como repositorio de notificaciones).

Debilidad que me pueden preguntar: la etapa de pruebas usa `npm run test:coverage || ... || echo "WARN"`, así que **una prueba roja no detiene el pipeline**; lo detiene el **Quality Gate** de SonarQube. Mejora propuesta: quitar el `|| echo` o agregar una etapa `npm run test:regression:bolsillos` sin tolerancia a fallos.

8. Preguntas probables y respuestas

¿Por qué el depósito como sexta funcionalidad? Revisé autores, ramas y pruebas del repo. Créditos es de Andrés; autenticación y perfil, de Tomás; tarjetas, de Dubin. Cuentas no tenía pruebas de nadie. Además el depósito alimenta el saldo disponible que usan los bolsillos, y lo pruebo con RG-DE-15.

¿Por qué Chai y no "Fluent Assertions"? `FluentAssertions` es de .NET. En JS/TS el estándar de aserciones fluidas es Chai BDD, y Vitest ya lo trae: no agregué dependencias.

¿Qué ganas con las aserciones fluidas? Se leen como una frase, agrupan condiciones sobre el mismo objeto en una sola cadena y dan mensajes de fallo más claros (`expected 201 to equal 400`).

¿Cambiate lo que verificaban las pruebas existentes? No, solo el estilo. Excepción: las de la entidad `Pocket`, que ahora también verifican tipo y código del error.

¿Por qué regresión a nivel API? Protege el comportamiento que consume el frontend (códigos HTTP, forma del JSON, mensajes de error). Una unitaria no detecta un cambio en el validador ni en el manejador de errores.

¿Por qué no Supabase real? Por repetibilidad y velocidad (≈ 4 s, sin red), y para no depender de la tabla `pockets`, que no existe (defecto D-07).

¿Qué son los 5 "expected fail"? Defectos conocidos: D-01/03 (nombre vacío), D-02 (booleano en bolsillos), D-04 (monto como texto al actualizar), D-05 (saldo contado dos veces) y D-08 (booleano en el depósito). Verifiqué que cada uno falla justo en la aserción del defecto.

¿Cómo sabes que la suite detecta errores? Por la demo: introduje cuatro errores distintos y todos se detectaron. El único cambio no detectado es un mutante equivalente, que no altera el comportamiento.

¿Qué encontraste en la sexta funcionalidad?

- El depósito acepta `true` como \$1 (D-08).
- Ignora la descripción y no registra una transacción.
- Usa códigos de error distintos a Bolsillos para el mismo problema.
- En `main` estaba mal cableado; la rama del pipeline lo corrigió.

¿Qué cobertura tienen? Los 9 archivos de las seis funcionalidades quedan al **100 %** de líneas, ramas y funciones. El backend completo, al 99,6 %.

9. Comandos para la sustentación

```
cd backend
npm run test:regression:bolsillos      # 6 funcionalidades: regresión + unitarias (198)
npx vitest run src/tests/regression    # solo la regresión de API (94)
npx vitest run src/tests/unit/account  # unitarias del depósito (12)
npx vitest run --reporter=verbose
src/tests/regression/account/depositar.regression.test.ts
npm run test:coverage                  # lo mismo que ejecuta Jenkins
```

10. Archivos para mostrar

Qué mostrar	Archivo
App de prueba (Bolsillos + depósito)	<code>backend/src/tests/helpers/createPocketTestApp.ts</code>
Escenario, catálogo de errores, invariante	<code>backend/src/tests/regression/pocket/support/scenarios.ts</code>

Qué mostrar	Archivo
Regresión por funcionalidad	backend/src/tests/regression/pocket/*.regression.test.ts y regression/account/depositar.regression.test.ts
Fakes con semántica de copia	backend/src/tests/fakes/InMemoryPocketRepository.ts
Fluent assertions (unitarias)	backend/src/tests/unit/pocket/CreatePocket.test.ts , unit/account/DepositMoney.test.ts
Evidencia antes/después	docs/evidencia-regresion-bolsillos/
Documento técnico	docs/pruebas-regresion-bolsillos.pdf
Pipeline	Jenkinsfile y repo FuBanking-gitops